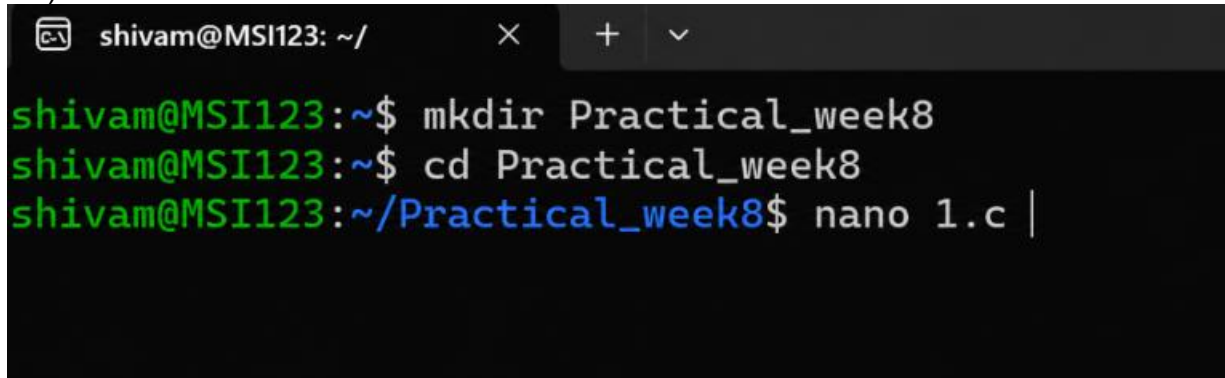


PRACTICAL WEEK 8

Task 1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

1) Create a file

A terminal window with a dark background. The title bar shows 'shivam@MSI123: ~/'. The terminal contains three lines of commands and their outputs: 'mkdir Practical_week8', 'cd Practical_week8', and 'nano 1.c'. The prompt changes from '~\$' to '~/Practical_week8\$' after the 'cd' command.

```
shivam@MSI123: ~$ mkdir Practical_week8
shivam@MSI123: ~$ cd Practical_week8
shivam@MSI123: ~/Practical_week8$ nano 1.c |
```

Then press enter to open command

```
shivam@MSH123: ~/Practical_week8 × + v
GNU nano 8.7.1 1.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    // malloc()
    int *a = (int *)malloc(3 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 3; i++)
    {
        a[i] = (i + 1) * 10;
    }

    printf("Memory allocated using malloc:\n");
    for (i = 0; i < 3; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");

    // calloc()
    int *b = (int *)calloc(3, sizeof(int));

    if (b == NULL)
    {
        printf("calloc failed\n");
        free(a);
        return 1;
    }
}
```

2)

write a code

```
shivam@MSI123: ~/Practical_week8 × + v
GNU nano 8.7.1 1.c *

printf("\nMemory allocated using calloc:\n");
for (i = 0; i < 3; i++)
{
    printf("%d ", b[i]);
}
printf("\n");

// realloc()
a = (int *)realloc(a, 5 * sizeof(int));

if (a == NULL)
{
    printf("realloc failed\n");
    free(b);
    return 1;
}

a[3] = 40;
a[4] = 50;

printf("\nMemory after realloc:\n");
for (i = 0; i < 5; i++)
{
    printf("%d ", a[i]);
}
printf("\n");

// free()
free(a);
free(b);

printf("\nMemory successfully freed.\n");

return 0;
}
```

- 1) To save press control O
- 2) To come back to shell press control X
- 3) Then DO compilation

```
shivam@MSI123: ~  
shivam@MSI123:~$ mkdir Practical_week8  
shivam@MSI123:~$ cd Practical_week8  
shivam@MSI123:~/Practical_week8$ nano 1.c  
shivam@MSI123:~/Practical_week8$ gcc 1.c -o 1  
shivam@MSI123:~/Practical_week8$ |
```

OUTPUT:

```
shivam@MSI123: ~/Practical_week8  
shivam@MSI123:~$ mkdir Practical_week8  
shivam@MSI123:~$ cd Practical_week8  
shivam@MSI123:~/Practical_week8$ nano 1.c  
shivam@MSI123:~/Practical_week8$ gcc 1.c -o 1  
shivam@MSI123:~/Practical_week8$ ./1  
Memory allocated using malloc:  
10 20 30  
  
Memory allocated using calloc:  
0 0 0  
  
Memory after realloc:  
10 20 30 40 50  
  
Memory successfully freed.  
shivam@MSI123:~/Practical_week8$ |
```

Task-2: Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
shivam@MSI123:~/Practical_week8$  
shivam@MSI123:~/Practical_week8$  
shivam@MSI123:~/Practical_week8$  
shivam@MSI123:~/Practical_week8$ nano 2.c |
```

CODE:

```
GNU nano 8.7.1 2.c *  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
#define SIZE (10 * 1024 * 1024)  
  
void print_memory(const char *message)  
{  
    FILE *file;  
    char line[256];  
  
    printf("\n--- %s ---\n", message);  
  
    file = fopen("/proc/self/status", "r");  
  
    if (file == NULL)  
    {  
        perror("fopen");  
        return;  
    }  
  
    while (fgets(line, sizeof(line), file))  
    {  
        if (strncmp(line, "VmSize:", 7) == 0 ||  
            strncmp(line, "VmRSS:", 6) == 0)  
        {  
            printf("%s", line);  
        }  
    }  
  
    fclose(file);  
}
```

```
}

int main()
{
    char *data;

    data = malloc(SIZE);

    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    // Touch the memory so that pages are actually allocated
    for (int i = 0; i < SIZE; i += 4096)
    {
        data[i] = 'A';
    }

    print_memory("Before fork()");

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild process created. PID = %d\n", getpid());
        print_memory("Child immediately after fork()");
    }
}
```

```
}

if (pid == 0)
{
    printf("\nChild process created. PID = %d\n", getpid());
    print_memory("Child immediately after fork()");
    printf("\nChild modifying memory...\n");

    // Modify every page
    for (int i = 0; i < SIZE; i += 4096)
    {
        data[i] = 'B';
    }

    print_memory("Child after modifying memory");
    sleep(5);
    free(data);
    exit(0);
}
else
{
    printf("\nParent process. PID = %d\n", getpid());
    print_memory("Parent after fork()");
    wait(NULL);
    free(data);
}

return 0;
}
```

OUTPUT:

```
shivam@MSI123:~/Practical_week8$  
shivam@MSI123:~/Practical_week8$  
shivam@MSI123:~/Practical_week8$ nano 2.c  
shivam@MSI123:~/Practical_week8$ gcc 2.c -o 2  
shivam@MSI123:~/Practical_week8$ ./2  
  
--- Before fork() ---  
VmSize:    13012 kB  
VmRSS:     11928 kB  
  
Parent process. PID = 1321  
  
--- Parent after fork() ---  
VmSize:    13012 kB  
VmRSS:     12056 kB  
  
Child process created. PID = 1322  
  
--- Child immediately after fork() ---  
VmSize:    13012 kB  
VmRSS:     11240 kB  
  
Child modifying memory...  
  
--- Child after modifying memory ---  
VmSize:    13012 kB  
VmRSS:     11240 kB
```